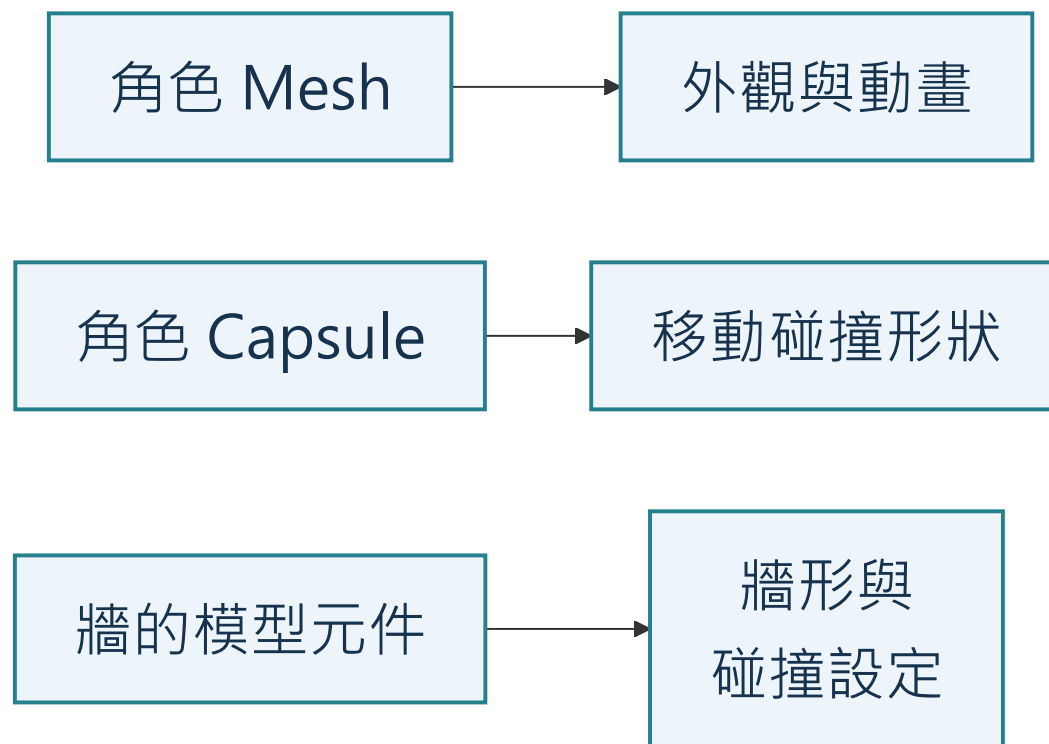


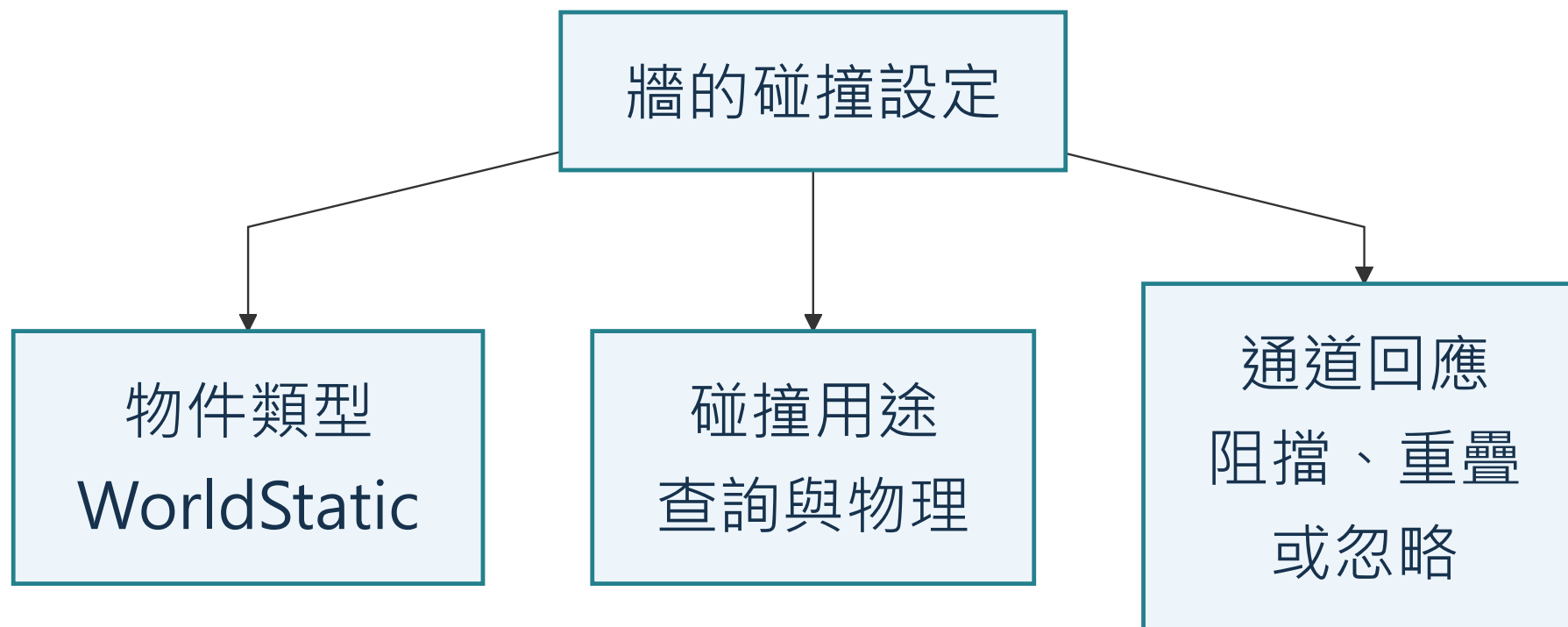
看得見的身體，不等於碰撞邊緣



先辨認是哪個元件負責碰撞，才知道該改哪個設定。

程式定位 | FightingPlayerCharacter.cpp、FightingCpuCharacter.cpp：各自建構子的 GetCapsuleComponent()->InitCapsuleSize()；
FightingCharacter.h：ACharacter 繼承宣告。

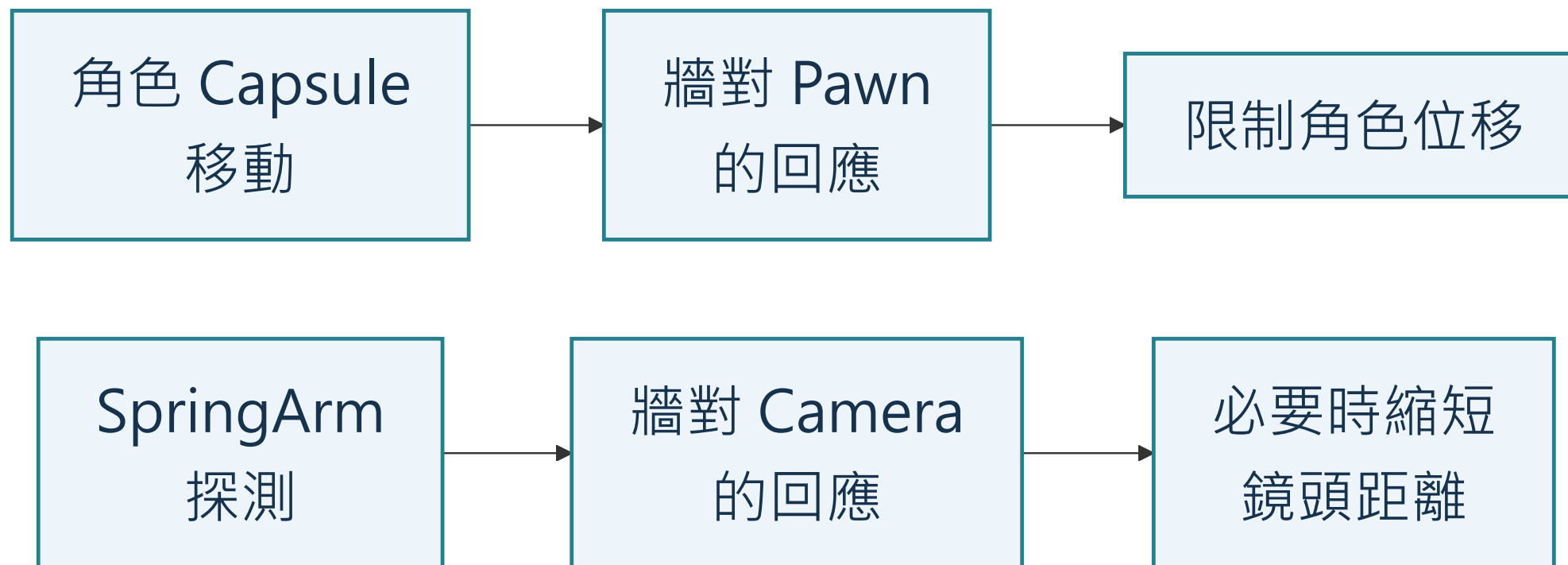
碰撞設定的三個問題



物件類型、碰撞用途與回應分開看；Preset 是它們的設定組合。

程式定位 | 無。此頁直接讀 stage1 的牆段元件設定，牆不另有場地限制 C++ 類別。

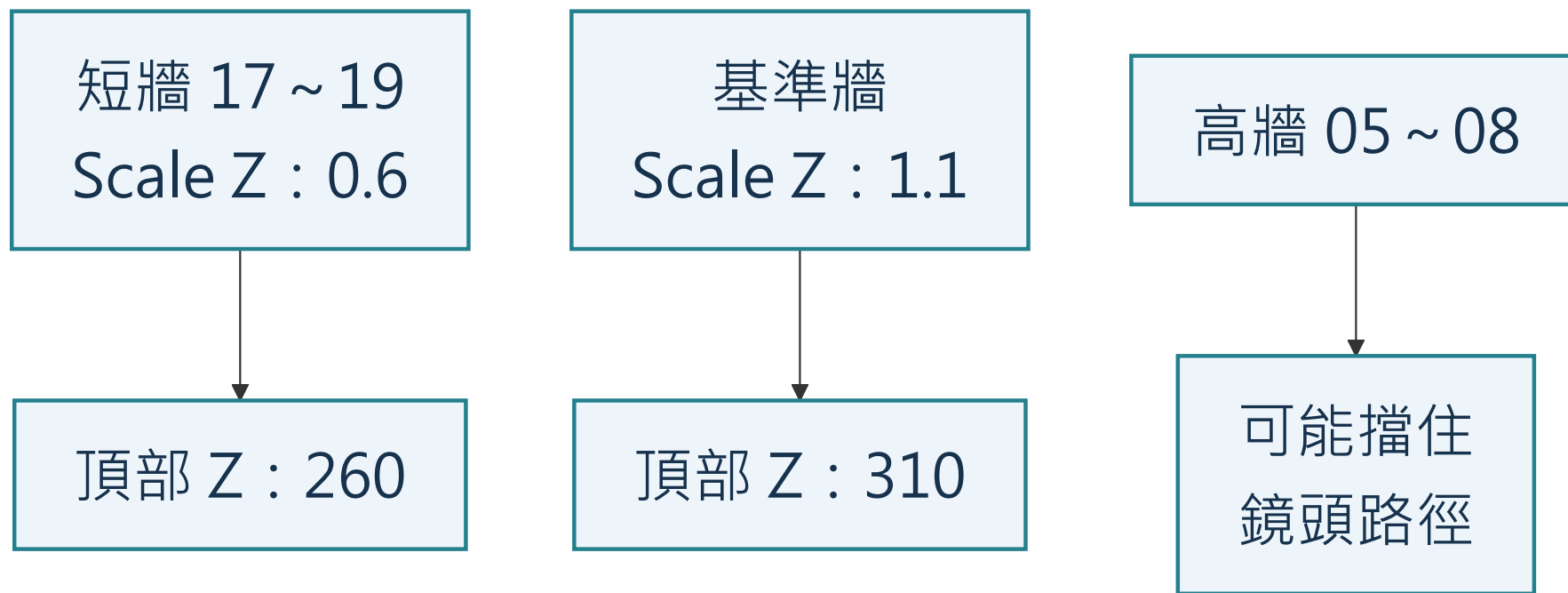
擋住角色與擋住鏡頭，是兩條路



同一面牆可以同時參與角色碰撞與鏡頭探測，結果各自影響不同對象。

程式定位 | FightingCameraActor.cpp : AFightingCameraActor()、CameraBoom->bDoCollisionTest、CameraBoom->ProbeChannel。

短牆、基準牆與高牆

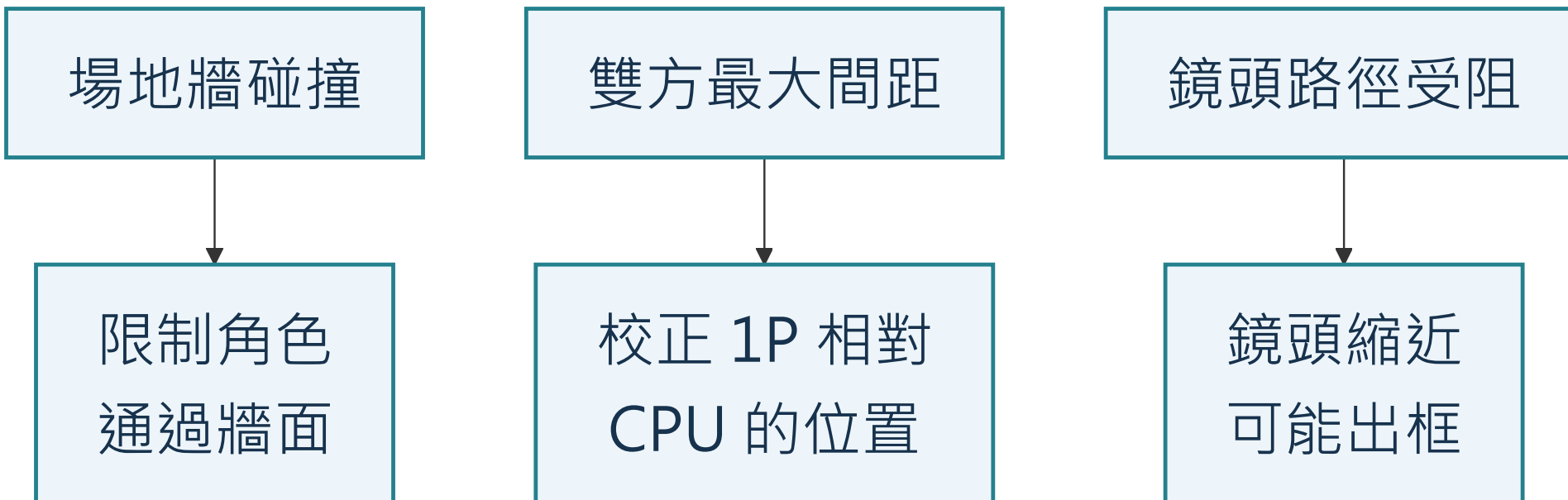


本教材 SM_Cube 高度為 100 cm，底部 Z=200。牆頂 $Z=200 + 100 \times \text{Scale Z}$ ，所以 $0.6 \rightarrow 260$ 、 $1.1 \rightarrow 310$ 。Scale 是倍率，Z 是世界高度；此式只適用於本題未傾斜、底角 Pivot 且位置不變的牆段。

牆形改變碰撞空間；只改高度時，要保留底部與平面接縫。

程式定位 | 無。此頁操作 stage1 既有牆段的 Transform。

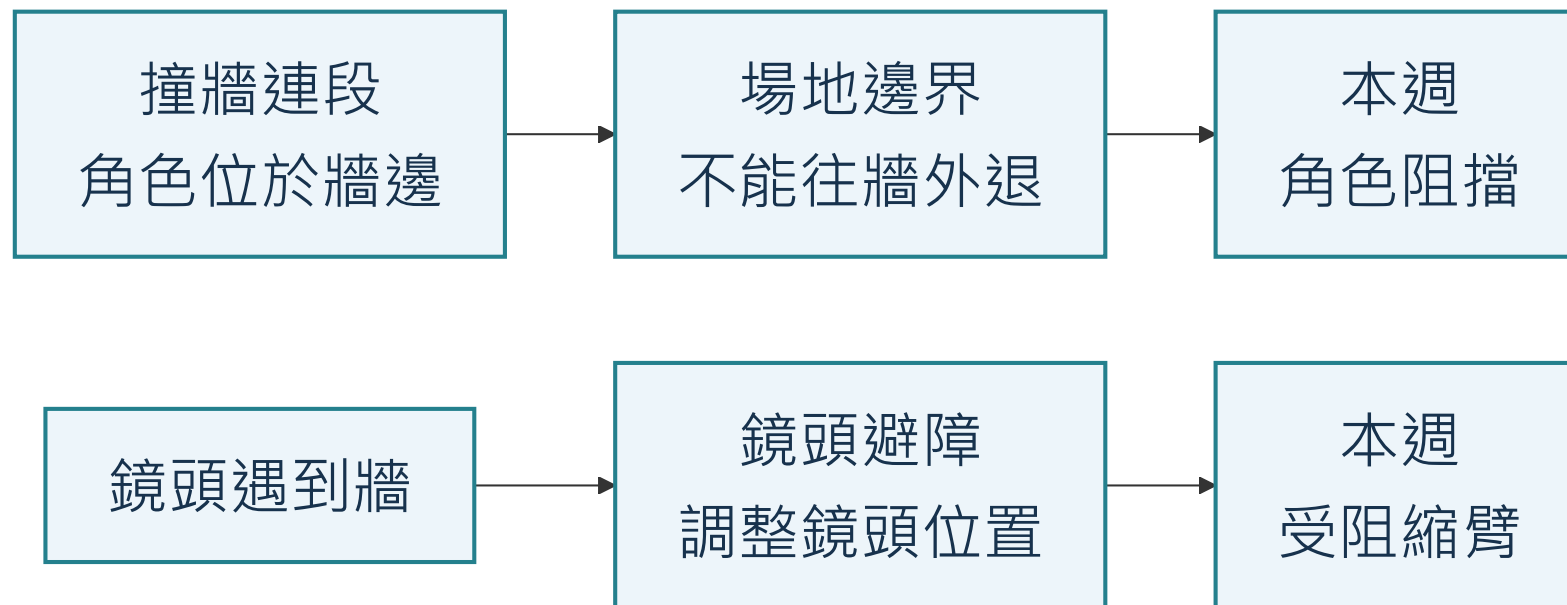
三種限制，分別看結果



角色停住或畫面變近時，先找出正在生效的規則。

程式定位 | FightingCameraActor.cpp : Tick()、ConstrainPlayerToMaxDistance()、CameraBoom->TargetArmLength ;
FightingCameraActor.h : MaxFighterDistance。

遊戲畫面對照：牆邊站位與鏡頭避牆



撞牆受擊、連段傷害不屬於本週。教材使用 SpringArm 受阻縮臂，仍可能出框；影片不作為教材設定的證據。

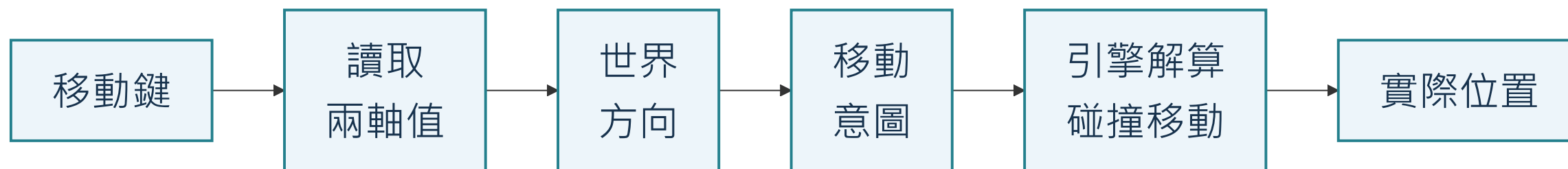
撞牆連段 (0:11) 開啟影片

格鬥鏡頭不能被牆阻擋 (2:00) 開啟影片

牆邊受擊與鏡頭避障都需要場地牆，但分別是遊戲規則與鏡頭規則。

程式定位 | 無。外部影片；回教材對照 FightingCameraActor.cpp : CameraBoom，以及 stage1 的場地牆。

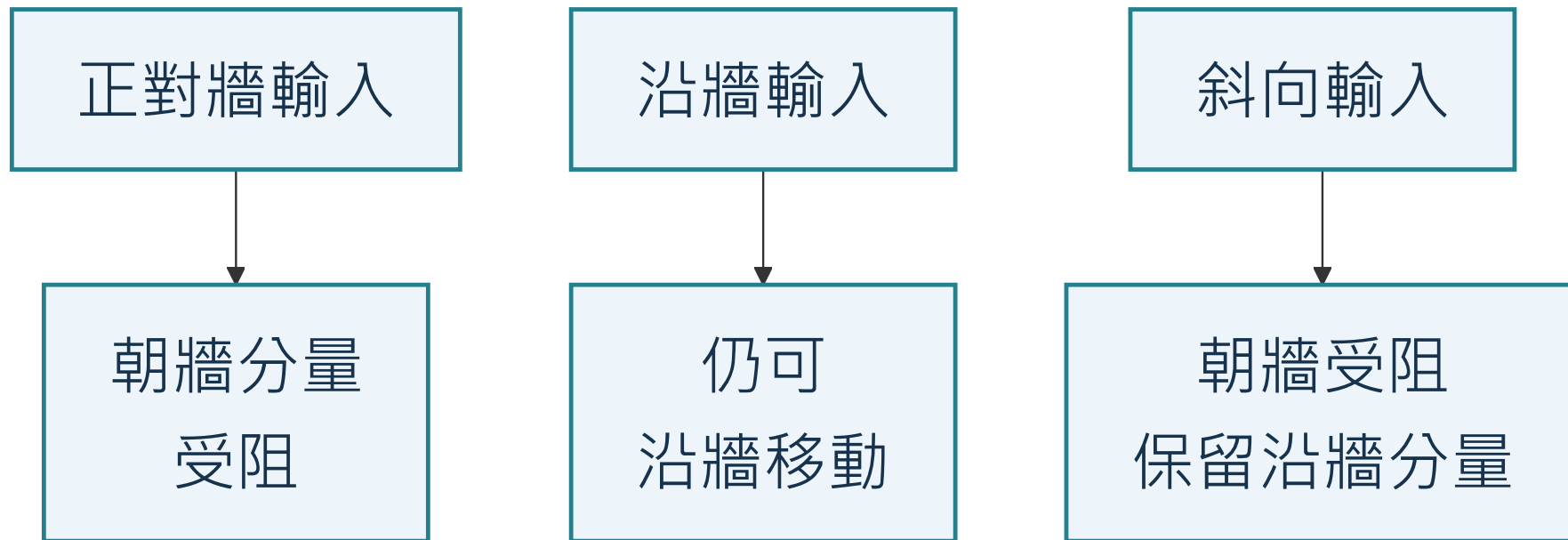
有輸入，不等於走完要求的距離



輸入描述想往哪裡走，位置才描述最後走到了哪裡。

程式定位 | FightingPlayerCharacter.cpp : Move()、DoMove()、兩次 AddMovementInput()。

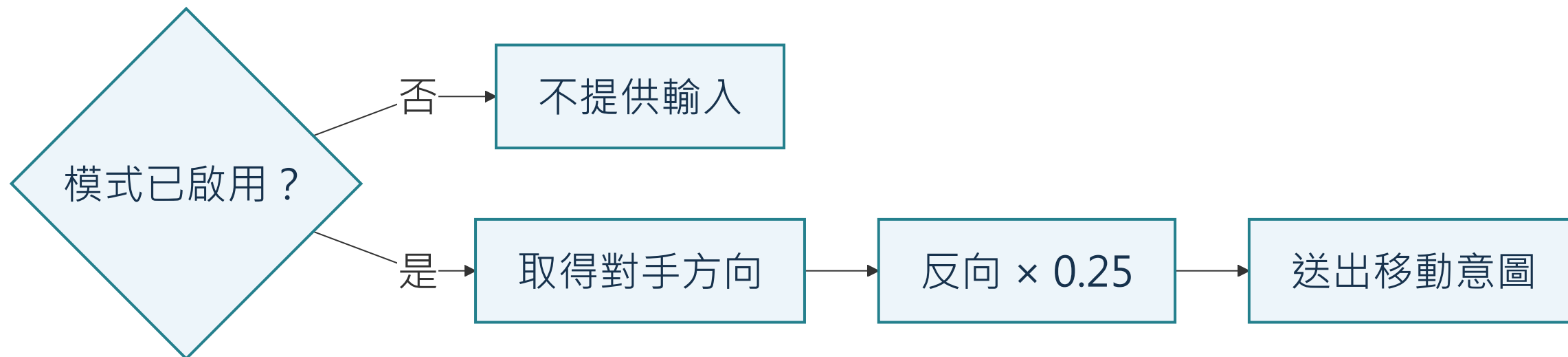
正對牆、沿牆與斜向



以牆面作基準看方向；斜向輸入仍可能保留沿牆的移動。

程式定位 | FightingPlayerCharacter.cpp : DoMove() 的 ForwardDirection、RightDirection 與 AddMovementInput()。碰撞後的移動由引擎 CharacterMovement 處理，此頁不逐行展開引擎內部實作。

CPU 後退模式持續送出什麼？



本圖只展開 RetreatFromPlayerSlow 且已有有效對手方向的分支；CPU 每幀執行，取不到有效方向時也不提供輸入。其他已啟用模式不一定使用反向或 0.25。

後退模式是持續的輸入來源，碰牆停止不表示模式被關閉。

程式定位 | FightingCpuCharacter.h : EFightingCpuTestMovementMode、TestMovementMode ; FightingCpuCharacter.cpp : Tick()、UpdateTestMovement()。

推擠要求走十公分，真的走了多少？

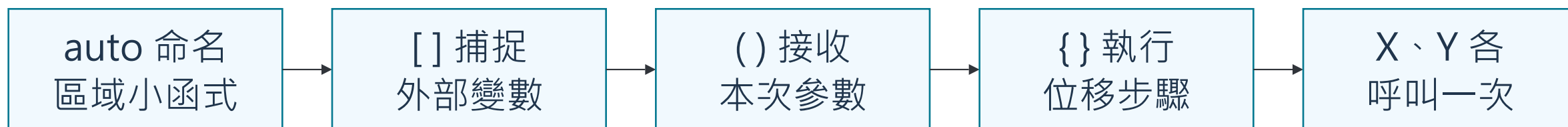


實際位移 = 移動後位置 - 移動前位置。例如同一水平軸的座標由 100 → 103 cm，完成量是 3 cm；原本要求 10 cm，不代表已走滿。本片段再把 Z 分量歸零，只保留地面位移。

要求位移可以被牆截短；用前後位置差確認實際完成量。

程式定位 | FightingPlayerCharacter.cpp : ResolveGroundPushInteraction() 內的 MovePairAlongAxis，只讀 PushedStart、RequestedOffset、AddActorWorldOffset()、AvailableOffset。

一段程式要用兩次：讀懂 C++ lambda



常見語法：[捕捉](參數) -> 回傳型別 { 內容 }; 回傳型別可省略。

[] 不捕捉 [x] 複製 x [&x] 參照 x

用途：在附近重用，或交給演算法 / 回呼；本例 X、Y 只更換 RequestedOffset。

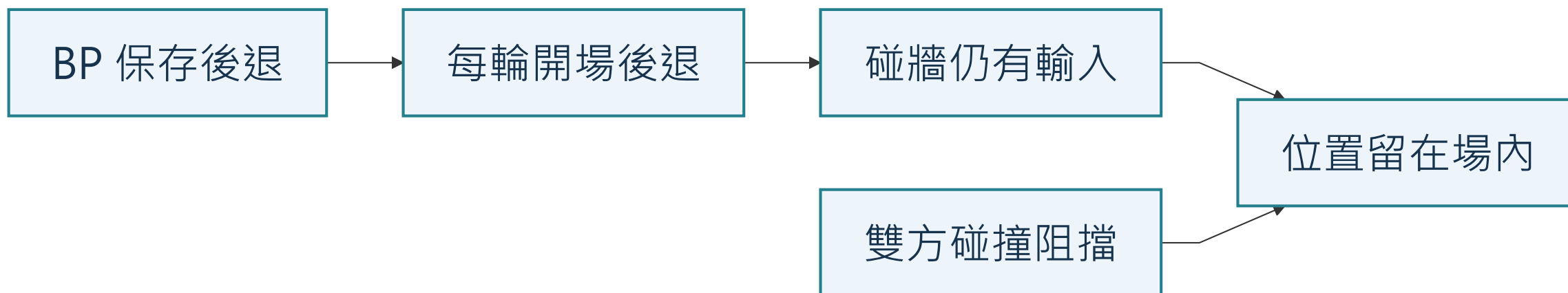
改成 private method 時，需傳入兩名角色與位移量；跨函式重用時可考慮抽出。

注意：參照捕捉或保存指標供回呼使用時，須確認呼叫當下對象有效。

捕捉固定角色；每次呼叫只更換軸向位移。

程式定位 | FightingPlayerCharacter.cpp：ResolveGroundPushInteraction() 內 MovePairAlongAxis 的宣告，以及 XOffset、YOffset 兩次呼叫。

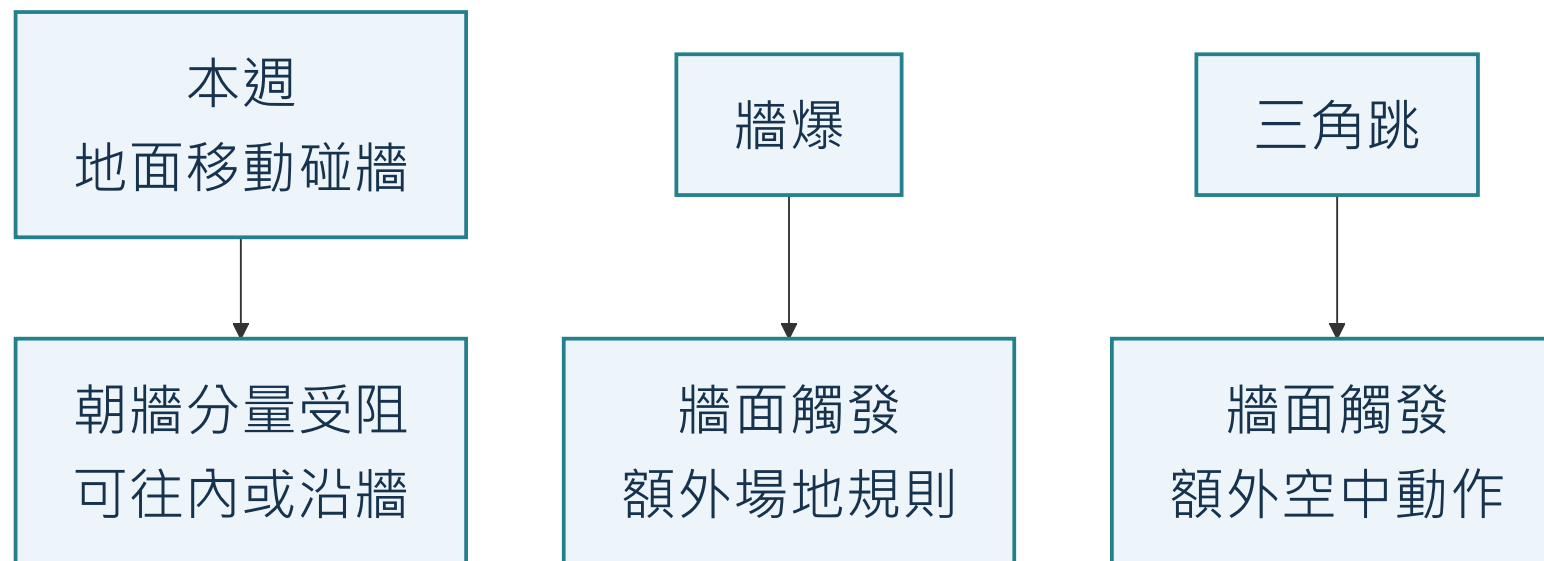
用三份證據驗收 CPU 碰牆



保存設定、確認輸入來源，再用實際位置證明受阻。

程式定位 | FightingCpuCharacter.h : TestMovementMode ; FightingCpuCharacter.cpp : UpdateTestMovement() ;
FightingCameraActor.cpp : ConstrainPlayerToMaxDistance()，僅在 1P 被校正時用來說明。

遊戲畫面對照：牆面可觸發其他玩法



本週沒有牆爆、牆體破壞或三角跳；沿牆移動仍是一般地面移動。影片只用來對照額外玩法。

牆爆 (0:28) [開啟影片](#)

三角跳 (0:02) [開啟影片](#)

碰撞可以只是阻擋，也可以成為其他遊戲規則的觸發條件；本週只實作阻擋。

程式定位 | 無。外部影片；回教材對照 FightingPlayerCharacter.cpp : DoMove()、FightingCpuCharacter.cpp : UpdateTestMovement()。